PROJECT REPORT

Sanskrit Document Retrieval-Augmented Generation (RAG) System

Domain	Platform	Language	Framework
NLP / AI	CPU-only	Python	LangChain

An end-to-end AI pipeline for question answering over Sanskrit documents, combines semantic information retrieval with generative language models.

1. Introduction

The Sanskrit Document Retrieval-Augmented Generation (RAG) System is an AI-based application designed to answer questions based on Sanskrit documents. It combines advanced information retrieval and language generation techniques to produce accurate and context-aware responses.

The system is built to serve researchers, educators, and enthusiasts working with Sanskrit-language content, providing intelligent question-answering capabilities without requiring deep technical expertise from end users.

A key design principle of this system is its compatibility with CPU-based inference, making it:

- Lightweight and memory-efficient
- Deployable in low-resource or offline environments
- Accessible without specialized GPU hardware

2. Objectives

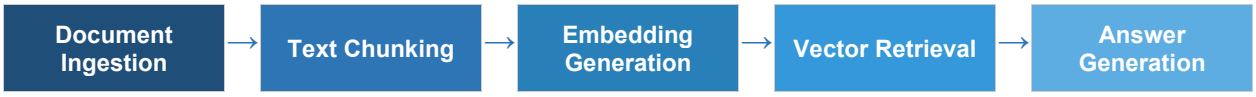
The primary objectives of this project are:

- Build a robust end-to-end RAG pipeline tailored for Sanskrit documents
- Extract and preprocess text from PDF documents in Sanskrit
- Retrieve contextually relevant information using semantic vector search
- Generate meaningful, natural-language answers using a Large Language Model (LLM)
- Ensure full compatibility with CPU-only execution environments

3. System Architecture

The system follows a standard RAG pipeline consisting of five core stages: document ingestion, text preprocessing, embedding generation, vector retrieval, and answer generation.

Pipeline Overview:



4. System Components

The system is composed of six modular components, each responsible for a distinct stage in the pipeline:

Component	Tool / Model	Description
Document Loader	PyPDFLoader (LangChain)	Extracts raw text from Sanskrit PDF documents for downstream processing
Text Preprocessor	RecursiveCharacterTextSplitter	Divides large text bodies into manageable, overlapping chunks for efficient retrieval
Embedding Model	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	Converts text chunks into high-dimensional numerical vectors; supports Sanskrit and 50+ languages
Vector Database	FAISS (Facebook AI Similarity Search)	Stores document embeddings and enables fast, scalable similarity search
Retrieval Module	FAISS-based Vector Similarity Search	Identifies and retrieves the most semantically relevant chunks for any given query
Language Model	FLAN-T5-Small (Google, HuggingFace Transformers)	Seq2Seq transformer that generates fluent, context-aware answers from retrieved passages

5. System Workflow

The complete end-to-end workflow proceeds as follows:

1. Load the Sanskrit PDF document from local storage
2. Extract full text content using PyPDFLoader
3. Split extracted text into smaller, overlapping chunks
4. Perform approximate nearest-neighbor search in FAISS
5. Index and store all embeddings in the FAISS vector store
6. Accept a natural-language query from the user
7. Convert the query into an embedding using the same model
8. Perform approximate nearest-neighbor search in FAISS
9. Retrieve the top-k most relevant context passages
10. Combine retrieved context with the query and pass it to the LLM
11. Generate and return the final natural-language answer

6. Technologies Used

Category	Technology / Library

Programming Language	Python 3.x
RAG Framework	LangChain
Vector Search	FAISS (Facebook AI Similarity Search)
Embeddings	Sentence Transformers (HuggingFace)
Language Model	HuggingFace Transformers (FLAN-T5)
Deep Learning	PyTorch (CPU-only mode)
PDF Extraction	PyPDFLoader

7. CPU Optimization

The system is designed and optimized to run entirely on CPU hardware, with no GPU dependency required. Key optimizations include:

- Lightweight multilingual MiniLM embedding model with low memory footprint
- Compact FLAN-T5-Small generative model optimized for inference speed
- FAISS flat index structure suitable for small to medium-sized document collections
- Efficient chunk-based text splitting to minimize peak memory usage
- No CUDA dependencies — runs on any standard laptop or server

8. Challenges & Mitigations

Challenge	Mitigation / Approach
Noisy OCR-generated Sanskrit text in PDFs	Applied text cleaning and normalization as a preprocessing step before chunking
Slow model downloads from HuggingFace Hub	Pre-cached models locally; documented offline setup instructions
Limited Sanskrit-specific NLP models	Used multilingual sentence-transformer model with Sanskrit support
Retrieval quality dependent on chunking	Tuned chunk size and overlap parameters experimentally for best recall

9. Results

The system successfully demonstrates a complete and functional RAG pipeline. Key outcomes include:

- Accurate retrieval of semantically relevant Sanskrit text passages
- Context-aware answer generation for Sanskrit-language queries
- Full end-to-end pipeline execution with no GPU requirement
- Consistent performance on CPU-only hardware configurations

9.1 Sample Interaction

Stage	Content
Input Query	सीता का? (Who is Sita?)
Retrieved Context	सीता रामस्य पत्नी अस्ति।
Generated Answer	सीता रामस्य पत्नी अस्ति। (Sita is Ram's wife.)

10. Future Improvements

The following enhancements are planned to further improve the system's accuracy, usability, and scalability:

- Integrate Sanskrit-specific embedding models for higher retrieval precision
- Improve OCR preprocessing pipeline to handle noisy Sanskrit PDF scans
- Fine-tune the LLM on a curated Sanskrit language corpus
- Develop a user-friendly web interface using Streamlit or FastAPI
- Implement hybrid retrieval combining BM25 (keyword) and FAISS (semantic) search
- Add support for multi-document corpora and cross-document reasoning

11. Conclusion

The Sanskrit Document RAG System represents a meaningful step toward making classical Sanskrit knowledge accessible through modern AI technology. By combining multilingual embeddings, FAISS-based vector retrieval, and sequence-to-sequence language generation, the system delivers accurate, context-aware answers from Sanskrit sources.

The system is modular, efficient, and fully compatible with CPU-only environments, making it accessible to a wide range of users without specialized hardware. It establishes a solid foundation

for future work in Sanskrit NLP, including richer embedding models, larger corpora, and improved user interfaces.